

LAPORAN PRAKTIKUM

PEMROGRAMAN BERBASIS OBJEK

Exception Handling dalam Java

NAMA	YOVI ANDRI DWI ANUGRAH
NIM	20240040197
Mata Kuliah	PBO Sesi 10

Exception adalah kejadian abnormal yang terjadi selama eksekusi program dan mengganggu alur normal instruksi. Java menggunakan model object-oriented untuk exception: setiap exception adalah sebuah objek yang merupakan turunan dari kelas Throwable.

Hierarki kelas Exception di Java:

- Throwable (root)
- Error: masalah serius yang tidak dapat ditangani (OutOfMemoryError, dll.)
- Exception: kondisi yang dapat ditangani program
- RuntimeException: Unchecked exception (tidak wajib ditangani)
- Checked Exception: Wajib ditangani dengan try-catch atau throws

Jenis Exception	Contoh	Kategori
ArithmeticException	Pembagian dengan nol	Unchecked
ArrayIndexOutOfBoundsException	Akses indeks di luar batas array	Unchecked
NullPointerException	Akses objek null	Unchecked
IOException	Error operasi I/O file	Checked
Custom Exception	Dibuat sendiri oleh programmer	Checked/Unchecked

Percobaan 1 `ArrayIndexOutOfBoundsException`

Program asli mencoba mengisi nilai ke indeks ke-5 dari array berukuran 5 (indeks 0–4), sehingga menyebabkan `ArrayIndexOutOfBoundsException` yang tidak tertangani dan program berhenti secara paksa.

Kode perbaikan:

```
public class Exception {
    public static void main(String[] args)
    {
        int a[] = new int[5];    try {
            a[5] = 100; // indeks 5 tidak valid (valid: 0-4)
        } catch (Exception e) {
            System.out.println("Terjadi pelanggaran
memory");    }    }
}
```

Output program: Terjadi pelanggaran memory

Analisa: Blok try membungkus baris yang berpotensi error. Ketika JVM mendeteksi akses indeks ilegal, ia melempar `ArrayIndexOutOfBoundsException` yang ditangkap oleh blok `catch(Exception e)`. Karena `Exception` adalah superclass dari semua checked dan unchecked exception (kecuali `Error`), ia dapat menangkap jenis exception apapun.

Percobaan 2 – `ArrayIndexOutOfBoundsException` dengan Loop

Program asli iterasi `i` dari 0 sampai 3, namun array `greeting` hanya memiliki 3 elemen (indeks 0–2). Ketika `i=3`, terjadi `ArrayIndexOutOfBoundsException`.

Kode perbaikan:

```
while (i < 4) {
    try {
        System.out.println(greetings[i]);
        i++;
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Resetting index value");
        i = 0; // reset i ke 0 → loop infinite!    }
}
```

Output program: Hello World! → No, I mean it! → HELLO WORLD! → Resetting index value → Hello World! → ... (loop tak terbatas)

Analisa: Percobaan ini menggunakan `catch(ArrayIndexOutOfBoundsException e)` yang lebih spesifik daripada `catch(Exception e)`. Ketika terjadi error pada indeks ke-3, blok catch mereset `i=0` sehingga loop dimulai ulang dari awal — program ini sebenarnya menghasilkan infinite loop sebagai demonstrasi behavior exception handling di dalam iterasi.

Percobaan 3 ArithmeticException (Pembagian dengan Nol)

Operasi `bil/0` menyebabkan `ArithmeticException` karena pembagian bilangan bulat dengan nol tidak diperbolehkan dalam Java.

Kode perbaikan dengan multiple catch:

```
try {
    System.out.println(bil / 0);
} catch (ArithmeticException e) {
    System.out.println("Terjadi Aritmatika error");
} catch (Exception e) {
    System.out.println("Ini menhandle error yang terjadi");
}
```

Output program: Terjadi Aritmatika error

Analisa: Java mengeksekusi blok catch dari atas ke bawah dan hanya menjalankan blok catch pertama yang cocok. Karena `ArithmeticException` adalah subclass dari `Exception`, `catch(ArithmeticException e)` harus diletakkan SEBELUM `catch(Exception e)`. Jika dibalik, compiler akan memberi error 'exception has already been caught'. Ini membuktikan aturan: tangkap exception yang lebih spesifik terlebih dahulu.

Percobaan 4 – Multiple Exception dalam Satu Try Block

Program memiliki dua potensi error: akses `b[3]` (array hanya 3 elemen, indeks 0–2) dan pembagian `bil/0`. Urutan exception dalam blok try menentukan exception mana yang pertama dilempar.

Versi awal menempatkan `b[3]` sebelum `bil/0`, sehingga `ArrayIndexOutOfBoundsException` yang terjadi lebih dahulu. Perbaikan menukar urutannya:

```
try {
    System.out.println(bil / 0); // ArithmeticException terjadi duluan
    System.out.println(b[3]);    // Tidak pernah dieksekusi
} catch (ArithmeticException e) { ... }
    catch (ArrayIndexOutOfBoundsException e) { ... }
catch (Exception e) { ... }
```

Output program: Terjadi Aritmatika error

Analisa: Ketika exception dilempar, eksekusi dalam blok try langsung berhenti dan loncat ke blok catch yang sesuai. Baris setelah pernyataan yang menyebabkan exception tidak pernah dieksekusi. Hal ini penting dalam perancangan logika program agar urutan statement dalam try block dipertimbangkan dengan cermat.

Percobaan 5 Metode Exception: getMessage(), printStackTrace()

Percobaan ini mendemonstrasikan metode-metode penting dari objek exception untuk mendapatkan informasi detail tentang error yang terjadi.

```
catch (ArithmeticException e) {  
    System.out.println(e.getMessage());    // '/' by zero'  
    e.printStackTrace();                  // ke stderr  
    e.printStackTrace(System.out);        // ke stdout  
}
```

Output program:

Pesan error: / by zero Info stack erase

java.lang.ArithmeticException: / by zero
at Exception5.main(Exception5.java:5)

Analisa: Kelas Exception menyediakan beberapa metode diagnostik: getMessage() mengembalikan pesan singkat penyebab error; printStackTrace() mencetak stack trace lengkap ke stderr (berguna untuk debugging); printStackTrace(PrintStream) memungkinkan stack trace diarahkan ke output stream tertentu seperti System.out. Informasi ini sangat berharga dalam proses debugging untuk melacak asal-usul exception.

Percobaan 6 – Penggunaan Kata Kunci throw

Percobaan ini menunjukkan cara melempar exception secara manual menggunakan kata kunci throw dari dalam sebuah method.

```
static void demo() {  
    NullPointerException t = new NullPointerException("Coba Throw");  
    throw t;  
    // Baris ini tidak pernah dieksekusi    System.out.println("Ini tidak  
    lagi dicetak");  
}
```

Output program: Ada pesan error: java.lang.NullPointerException: Coba Throw

Analisa: Kata kunci throw digunakan untuk secara eksplisit melempar objek exception. Perlu diperhatikan bahwa baris kode setelah throw tidak akan pernah dieksekusi (unreachable code). Dalam contoh ini, kode 'System.out.println(...)' setelah throw bahkan akan menyebabkan compile error 'unreachable statement'. Exception yang dilempar dari method demo() akan merambat ke caller (main) dan ditangkap oleh blok catch di sana.

Percobaan 7 Exception dengan Pesan Kustom

Percobaan ini mendemonstrasikan cara membuat exception dengan pesan kustom dan menggunakan berbagai metode untuk menampilkan informasi exception tersebut.

```
try {    throw new Exception("Here's my Exception");
} catch (Exception e) {
    System.out.println(e.getMessage()); // Here's my Exception
    System.out.println(e.toString());   // java.lang.Exception: Here's my
Exception
    e.printStackTrace();
}
```

Output program: Caught

Exception

e.getMessage(): Here's my Exception

e.toString(): java.lang.Exception: Here's my Exception

e.printStackTrace(): java.lang.Exception: Here's my Exception at ThrowExample2.main(...)

Analisa: Perbedaan ketiga metode: getMessage() hanya mengembalikan pesan yang diberikan saat konstruksi exception; toString() mengembalikan nama kelas exception diikuti pesan (format: ClassName: message); printStackTrace() menampilkan informasi terlengkap termasuk call stack yang menunjukkan alur pemanggilan method hingga titik exception terjadi.

Percobaan 8 – Kata Kunci throws dan Blok finally

Kata kunci throws pada deklarasi method menandakan bahwa method tersebut dapat melempar exception tanpa menanganinya sendiri, dan tanggung jawab penanganan diserahkan kepada pemanggil method tersebut.

```
public void methodB() throws IOException {    System.out.println(20 / 0);
// ArithmeticException, bukan IOException!
}
```

Versi perbaikan menggunakan try-catch-finally di pemanggil:

```
try {
    o.methodB();
} catch (Exception e) {
    System.out.println("Error di Method B");
} finally {
    System.out.println("Ini selalu dicetak");
}
```

Output program: Method A → Error di Method B → Ini selalu dicetak

—

Analisa: Blok finally sangat penting karena selalu dieksekusi tanpa memandang apakah exception terjadi atau tidak, bahkan jika ada return statement di blok try atau catch. Ini berguna

untuk pembersihan sumber daya (menutup file, koneksi database, dll.). Catatan: method `methodB()` dideklarasikan `throws IOException` namun sebenarnya melempar `ArithmeticException` (`RuntimeException/unchecked`), sehingga tidak benar-benar memerlukan deklarasi `throws`.

Percobaan 9 – Exception Propagation dengan finally

Program ini menunjukkan bagaimana exception merambat (propagate) melalui call stack dan peran blok `finally`.

```
public static String reverse(String s) throws Exception
{
    if (s.length() == 0) {
        throw new Exception(); //
    }
    String kosong = exception;
    // Balik string secara manual
    String reverseStr = "";
    for (int i = s.length()-1; i >= 0; --i) {
        reverseStr += s.charAt(i);
    }
    return reverseStr;
}
```

Output dengan input 'This is a string': gnirts a si sihT → All done

Output dengan input " (kosong): The String was blank → All done

Analisa: Ketika string tidak kosong, method `reverse()` berhasil mengembalikan string yang dibalik. Blok `finally` tetap dieksekusi mencetak 'All done'. Ketika string kosong, method melempar `Exception` yang merambat ke `main()` dan ditangkap oleh `catch`. Ini mendemonstrasikan konsep exception propagation: exception yang tidak ditangani dalam suatu method akan otomatis merambat ke pemanggilnya.

Percobaan 10 – IOException dengan RandomAccessFile

Percobaan ini menggunakan `IOException` untuk menangani operasi file dengan kelas `RandomAccessFile` dari paket `java.io`.

```
try {
    RandomAccessFile books = new RandomAccessFile("books.txt", "rw");
    for (int i = 0; i < 3; i++) {
        books.writeUTF(bookList[i]);
        books.writeInt(yearList[i]);
    }
    books.seek(0);
    System.out.println(books.readUTF() + " " + books.readInt());
    books.close();
} catch (IOException e) {
    System.out.println("Indeks melebihi batas");
}
```

Output program: Satu 1920 → Dua 1230 → test

Analisa: RandomAccessFile memungkinkan akses file secara acak (non-sequential). Mode 'rw' berarti file dibuka untuk baca dan tulis. IOException adalah checked exception yang wajib ditangani karena operasi I/O dapat gagal karena berbagai alasan: file tidak ditemukan, izin akses ditolak, disk penuh, dll. Metode seek(0) memindahkan pointer file kembali ke posisi awal sebelum membaca. Pesan catch 'Indeks melebihi batas' sebenarnya kurang tepat karena IOException tidak selalu berkaitan dengan indeks.

Percobaan 11 – Custom Exception dengan extends Throwable

Percobaan ini menunjukkan cara membuat kelas exception sendiri dengan mewarisi langsung dari kelas Throwable.

```
class RangeErrorException extends Throwable {
    public RangeErrorException(String s) {
        super(s); // Teruskan pesan ke Throwable
    }

    // Penggunaan:
    if (position > 0) { throw new
        RangeErrorException("Position " + position);
    }
```

Output program: Range error: Position 1 → This is the last program.

Analisa: Dengan extends Throwable, kita mendapatkan fleksibilitas penuh namun ini bukan praktik terbaik. Umumnya disarankan extends Exception (untuk checked exception) atau extends RuntimeException (untuk unchecked exception). Konstruktor memanggil super(s) untuk meneruskan pesan ke induk kelas, sehingga getMessage() dapat bekerja. Blok catch hanya menangkap RangeErrorException, sehingga exception jenis lain tidak tertangkap.

Percobaan 12 – Custom Exception dengan extends Exception

Percobaan ini menunjukkan implementasi custom exception yang lebih umum digunakan, yaitu dengan mewarisi dari kelas Exception.


```

class MyException extends Exception {
    private String Teks;
    MyException(String s) {
        Teks = "Exception generated by: " + s;
        System.out.println(Teks);
    }
}

static void tampil(String s) throws Exception {

    System.out.println("Tampil");
    if (s.equals("amir")) {
        throw new MyException(s);
    }
    System.out.println("OK!");
}

```

Output program:

Tampil → OK! → Tampil → Exception generated by: amir → Tangkap: MyException Analisa: Pemanggilan tampil('ali') berhasil karena s tidak sama dengan 'amir', sehingga mencetak 'Tampil' dan 'OK!'. Pemanggilan tampil('amir') mencetak 'Tampil', lalu membuat dan melempar MyException baru. Konstruktor MyException mencetak pesan sebelum exception benar-benar dilempar ke pemanggil. Di main(), blok catch menangkap MyException dan mencetak objek exception menggunakan toString(). Ini contoh yang baik untuk domainspecific exception.